Green University of Bangladesh
Department of Computer Science and Engineering (CSE)

Faculty of Sciences and Engineering
Semester: (Fall, Year: 2025), B.Sc. in CSE (Day)

LAB REPORT NO: 05
Course Title: Operating System Lab
Course Code: CSE-402 Section: 231-D2

Student Details

Name		ID
1.	Promod Chandra Das	231002005

Lab Date :24-12-2025

Submission Date: 30-12-25

Course Teacher's Name : Md. Shoab Alam

[For Teachers use only: [Don't Write Anything inside this box]

Lab Report Status

Marks: …… **Signature: .....**
Comments: ..... **Date: .....**

1.TITLE OF THE LAB EXPERIMENT

Comparison Analysis of FIFO, LRU and OPR

2.OBJECTIVES

1.Understand why **page replacement** is needed in a **virtual memory** system.

2. Implement **FIFO, LRU**, and **Optimal (OPR)** page replacement algorithms.

3.Compute and compare:

- **Page Hits**
- **Page Faults**
- **Fault Rate**

4.Analyze behavior under different reference strings and frame sizes.

5.Observe key properties:

- FIFO can show **Belady's anomaly**
- LRU is a practical approximation to optimal
- OPR gives **the minimum possible faults** for a given reference string

3.PROCEDURE

1.Take input:

- Number of frames F
- Page reference string (space-separated integers)

2.Initialize frames as empty.

3. For each page request:

- If page is already in frames → **Hit**
- Else → **Fault**
  - If frame empty exists: place page there
  - Else choose victim using FIFO / LRU / OPR

4. Track:

- total hits, total faults
- optional: print frame state after each reference

5.Repeat for all 3 algorithms.

6.Compare fault counts and fault rates.

4.IMPLEMENTATION

Here is Bash Code: Input

```
 #!/usr/bin/env bash
# Comparison Analysis of FIFO, LRU and OPR (Optimal) Page Replacement
# Works on Ubuntu bash. Step-by-step frames + final hits/faults.

read -rp "Enter number of frames: " FRAMES
read -rp "Enter reference string (space-separated): " REFSTR

# Validate FRAMES
if ! [[ "$FRAMES" =~ ^[0-9]+$ ]] || [ "$FRAMES" -le 0 ]; then
  echo "Error: number of frames must be a positive integer."
  exit 1
fi

# Convert reference string into array
read -r -a REF <<< "$REFSTR"
N=${#REF[@]}
if [ "$N" -eq 0 ]; then
  echo "Error: reference string cannot be empty."
  exit 1
fi

print_line() {
  printf "%s\n" "--------------------------------------------------------------------------"
}

frames_to_string() {
  # usage: frames_to_string arrName
  local name="$1"
  local s="["
  local i val
  for ((i=0; i<FRAMES; i++)); do
    eval 'val="${'"$name"'[i]}"'
    if [ -z "$val" ]; then
```

```
    s+=" _"
   else
    s+=" $val"
   fi
  done
  s+=" ]"
  echo "$s"
}

# Returns 0 if page exists in frames array; 1 otherwise.
in_frames() {
  local page="$1"
  local name="$2"
  local i val
  for ((i=0; i<FRAMES; i++)); do
   eval 'val="${"$name"[i]}"'
   if [ "$val" = "$page" ]; then
    return 0
   fi
  done
  return 1
}

# Finds index of page in frames array, echoes index or -1
find_index() {
  local page="$1"
  local name="$2"
  local i val
  for ((i=0; i<FRAMES; i++)); do
   eval 'val="${"$name"[i]}"'
   if [ "$val" = "$page" ]; then
    echo "$i"
    return
   fi
  done
  echo "-1"
}

# Finds first empty slot index, echoes index or -1
find_empty() {
  local name="$1"
  local i val
  for ((i=0; i<FRAMES; i++)); do
   eval 'val="${"$name"[i]}"'
   if [ -z "$val" ]; then
    echo "$i"
    return
   fi
  done
  echo "-1"
```

```
}

print_header() {
  local algo="$1"
  echo
  print_line
  echo "Algorithm: $algo"
  print_line
  printf "%-6s %-6s %-32s %-10s\n" "Step" "Page" "Frames" "Result"
  print_line
}

print_footer() {
  local algo="$1" hits="$2" faults="$3"
  print_line
  echo "$algo Total References: $N"
  echo "$algo Hits:   $hits"
  echo "$algo Faults: $faults"
  awk -v f="$faults" -v n="$N" 'BEGIN { printf "%s Fault Rate: %.2f%%\n", (f/n)*100 }'
}

# ---------- FIFO ----------
run_fifo() {
  local -a fr
  local i
  for ((i=0; i<FRAMES; i++)); do fr[i]=""; done

  local faults=0 hits=0
  local ptr=0

  print_header "FIFO"

  local step p res empty
  for ((step=0; step<N; step++)); do
    p="${REF[step]}"
    res="HIT"

    if in_frames "$p" fr; then
      hits=$((hits+1))
    else
      res="FAULT"
      faults=$((faults+1))

      empty=$(find_empty fr)
      if [ "$empty" -ne -1 ]; then
        fr[$empty]="$p"
      else
        fr[$ptr]="$p"
        ptr=$(( (ptr + 1) % FRAMES ))
      fi
```

```
    fi

    printf "%-6s %-6s %-32s %-10s\n" "$((step+1))" "$p" "$(frames_to_string fr)" "$res"
  done

  print_footer "FIFO" "$hits" "$faults"
}

# ---------- LRU ----------
run_lru() {
  local -a fr last_used
  local i
  for ((i=0; i<FRAMES; i++)); do
    fr[i]=""
    last_used[i]=-1
  done

  local faults=0 hits=0

  print_header "LRU"

  local step p res idx empty victim min
  for ((step=0; step<N; step++)); do
    p="${REF[step]}"
    res="HIT"

    idx=$(find_index "$p" fr)
    if [ "$idx" -ne -1 ]; then
      hits=$((hits+1))
      last_used[$idx]="$step"
    else
      res="FAULT"
      faults=$((faults+1))

      empty=$(find_empty fr)
      if [ "$empty" -ne -1 ]; then
        fr[$empty]="$p"
        last_used[$empty]="$step"
      else
        # victim = least recently used => minimum last_used
        victim=0
        min="${last_used[0]}"
        for ((i=1; i<FRAMES; i++)); do
          if [ "${last_used[$i]}" -lt "$min" ]; then
            min="${last_used[$i]}"
            victim=$i
          fi
        done
        fr[$victim]="$p"
        last_used[$victim]="$step"
```

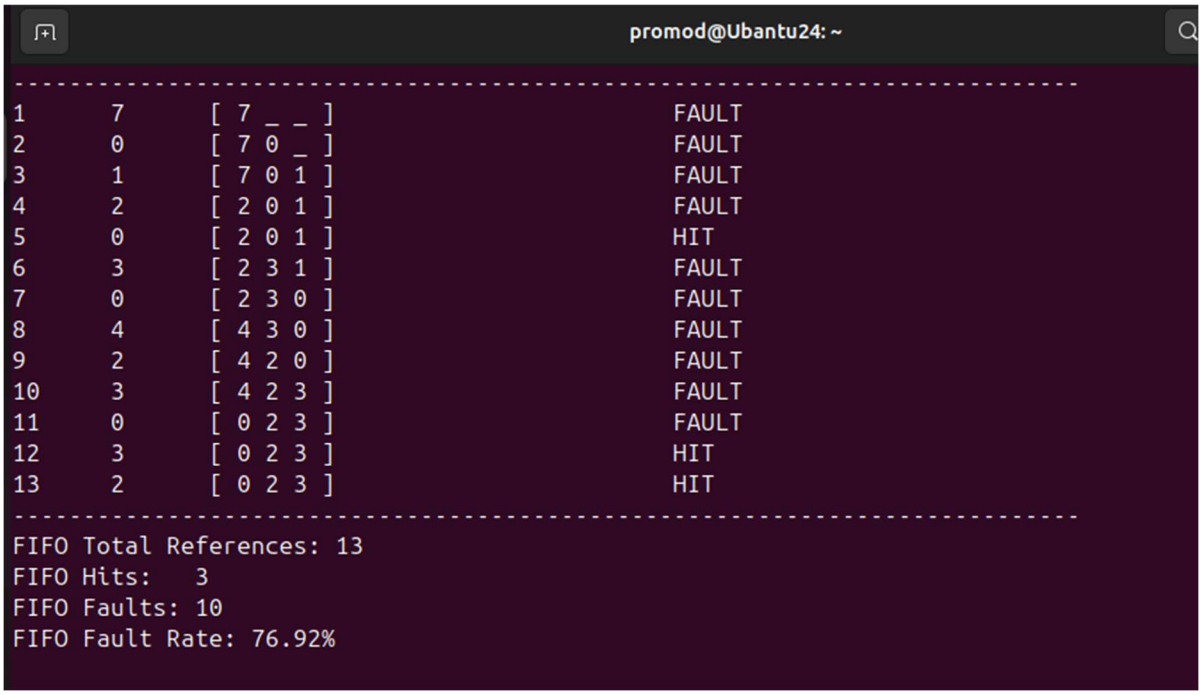
```
    fi
   fi

   printf "%-6s %-6s %-32s %-10s\n" "$((step+1))" "$p" "$(frames_to_string fr)" "$res"
  done

  print_footer "LRU" "$hits" "$faults"
}

# ---------- OPR (Optimal) ----------
run_opr() {
  local -a fr
  local i
  for ((i=0; i<FRAMES; i++)); do fr[i]=""; done

  local faults=0 hits=0

  print_header "OPR (Optimal)"

  local step p res empty victim farthest cur next j
  for ((step=0; step<N; step++)); do
   p="${REF[step]}"
   res="HIT"

   if in_frames "$p" fr; then
    hits=$((hits+1))
   else
    res="FAULT"
    faults=$((faults+1))

    empty=$(find_empty fr)
    if [ "$empty" -ne -1 ]; then
     fr[$empty]="$p"
    else
     victim=-1
     farthest=-1

     # choose page with farthest next use; if never used again, replace it
     for ((i=0; i<FRAMES; i++)); do
      cur="${fr[$i]}"
      next=-1
      for ((j=step+1; j<N; j++)); do
       if [ "${REF[$j]}" = "$cur" ]; then
        next=$j
        break
       fi
      done

      if [ "$next" -eq -1 ]; then
       victim=$i
```

```
        farthest=999999
        break
      fi

      if [ "$next" -gt "$farthest" ]; then
        farthest=$next
        victim=$i
      fi
    done

    fr[$victim]="$p"
  fi
 fi

  printf "%-6s %-6s %-32s %-10s\n" "$((step+1))" "$p" "$(frames_to_string fr)" "$res"
 done

 print_footer "OPR (Optimal)" "$hits" "$faults"
}

# Run all
run_fifo
run_lru
run_opr
```

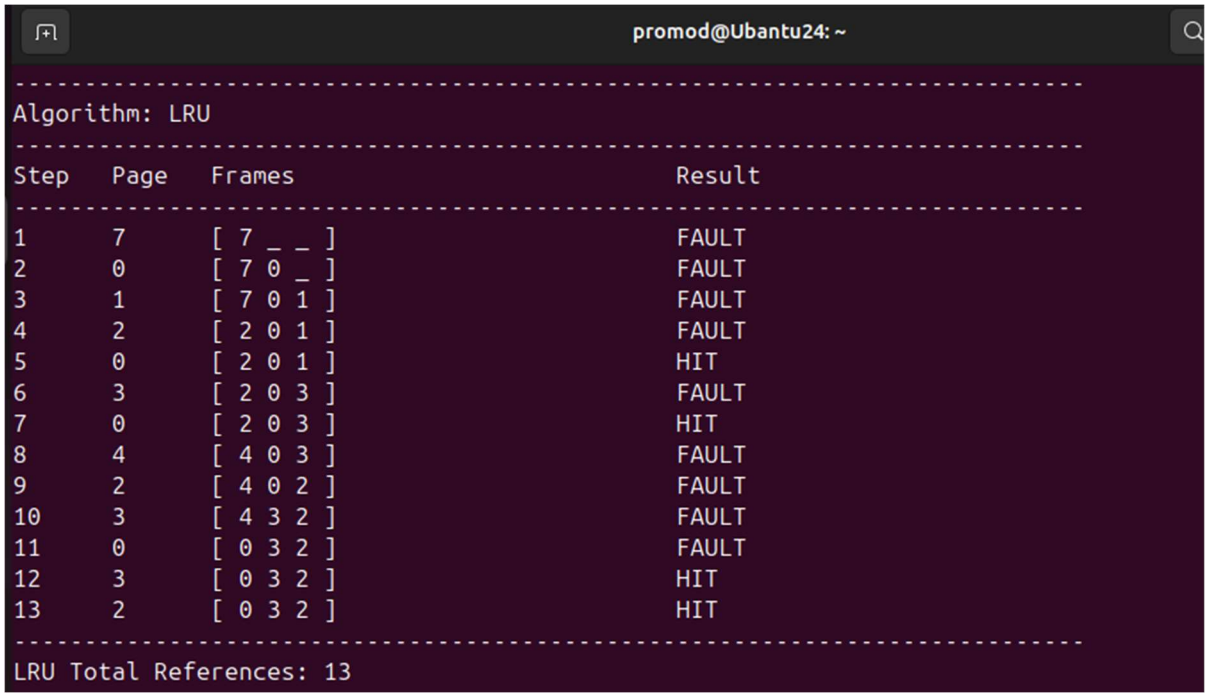
5.Output

```
promod@Ubantu24:~$ ./LabReport5.sh
Enter number of frames: 3
Enter reference string (space-separated): 7 0 1 2 0 3 0 4 2 3 0 3 2

--------------------------------------------------------------------------
Algorithm: FIFO
--------------------------------------------------------------------------
Step   Page   Frames                          Result
--------------------------------------------------------------------------
1      7      [ 7 _ _ ]                       FAULT
2      0      [ 7 0 _ ]                       FAULT
3      1      [ 7 0 1 ]                       FAULT
4      2      [ 2 0 1 ]                       FAULT
5      0      [ 2 0 1 ]                       HIT
6      3      [ 2 3 1 ]                       FAULT
7      0      [ 2 3 0 ]                       FAULT
8      4      [ 4 3 0 ]                       FAULT
9      2      [ 4 2 0 ]                       FAULT
10     3      [ 4 2 3 ]                       FAULT
11     0      [ 0 2 3 ]                       FAULT
```

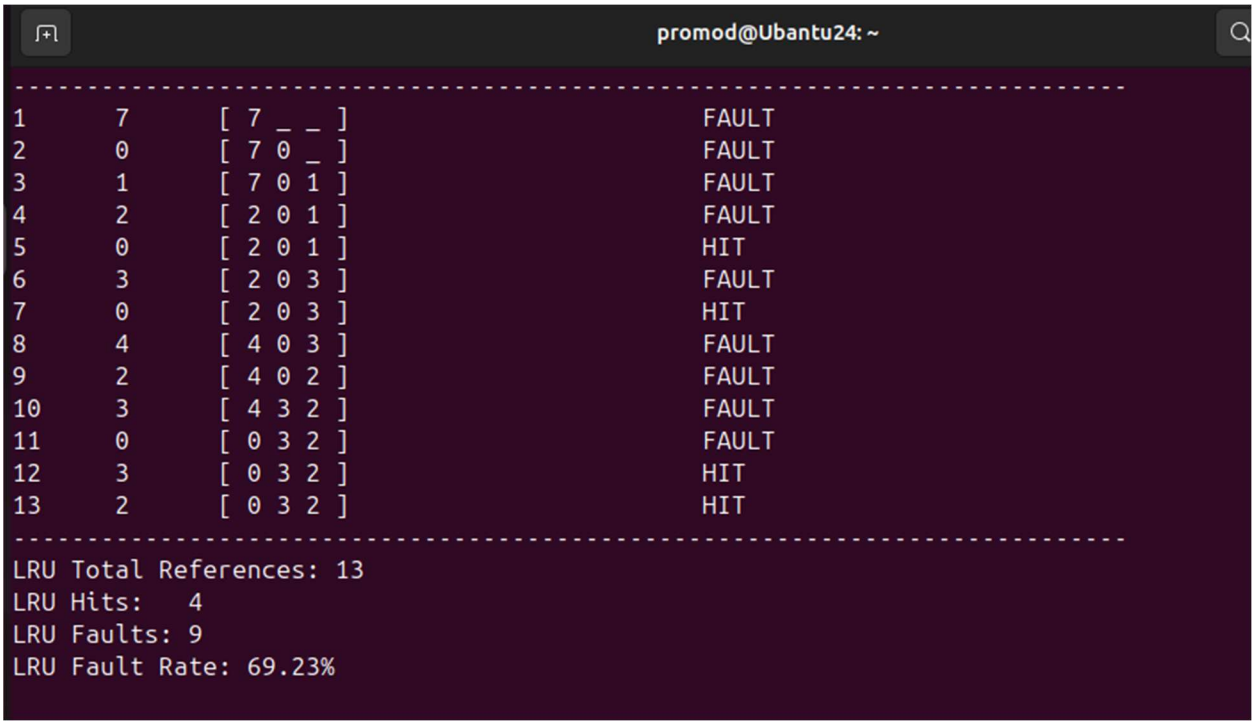
Fiqure 1: FIFO



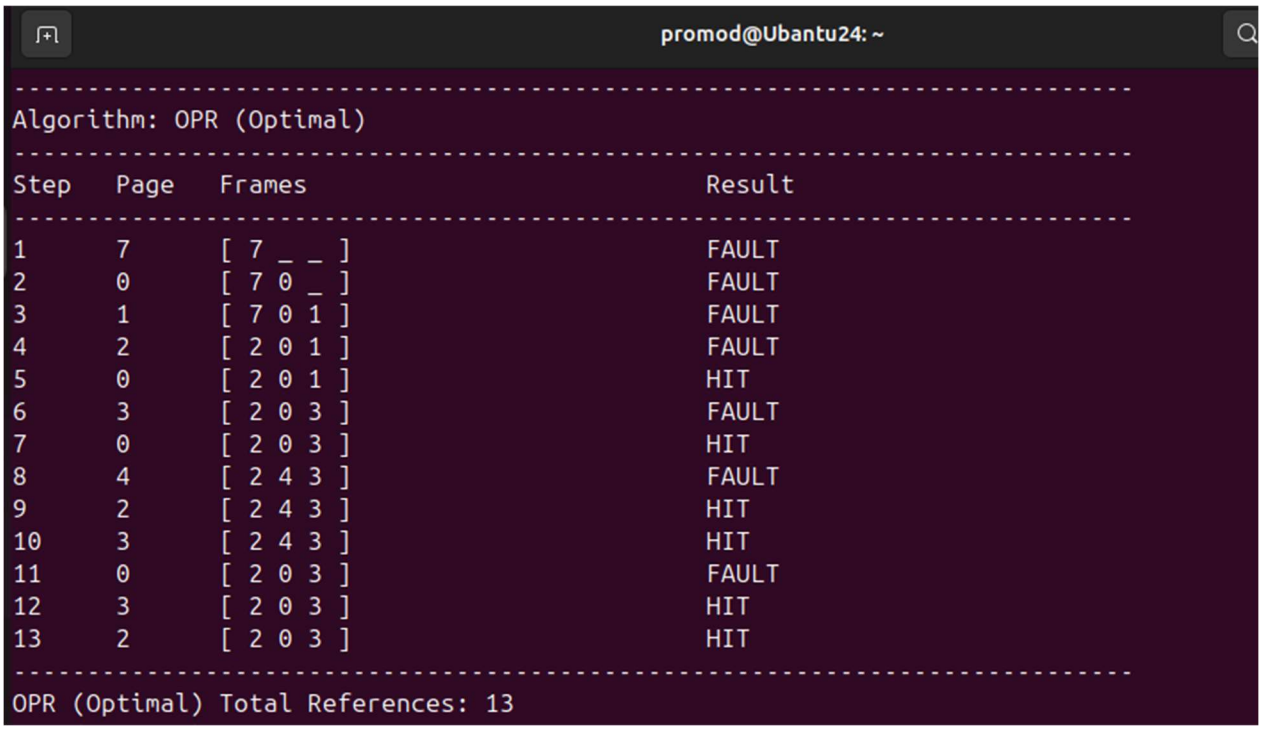
Fiqure 2: FIFO



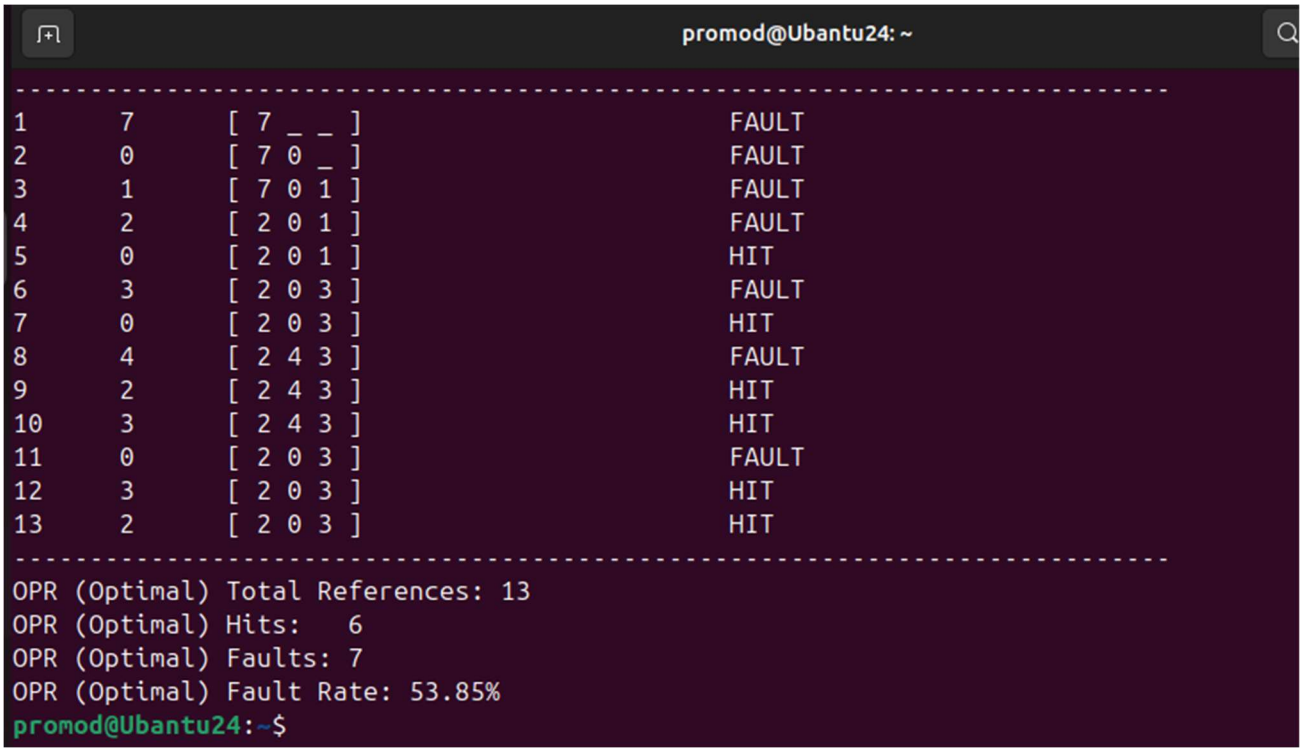
Fiqure 3: LRU



Fiqure 4: LRU



Fiqure 5: OPR


promod@Ubantu24: ~
1 7 [7 _ _] FAULT
2 0 [7 0 _] FAULT
3 1 [7 0 1] FAULT
4 2 [2 0 1] FAULT
5 0 [2 0 1] HIT
6 3 [2 0 3] FAULT
7 0 [2 0 3] HIT
8 4 [2 4 3] FAULT
9 2 [2 4 3] HIT
10 3 [2 4 3] HIT
11 0 [2 0 3] FAULT
12 3 [2 0 3] HIT
13 2 [2 0 3] HIT
OPR (Optimal) Total References: 13
OPR (Optimal) Hits: 6
OPR (Optimal) Faults: 7
OPR (Optimal) Fault Rate: 53.85%
promod@Ubantu24:~$

Fiqure 6: OPR

6.ANALYSIS AND DISCUSSION

❖ **Comparison Points**

1. **Fault Minimization**
    - **OPR** gives the minimum possible page faults for a given reference string and frame count because it uses future knowledge.
    - **LRU** approximates OPR using past behavior and performs well because recent usage often predicts near-future usage.
    - **FIFO** ignores usage patterns and can evict frequently used pages → higher faults.
2. **Implementation Complexity**
    - FIFO: simplest (queue pointer)
    - LRU: moderate (track last-used timestamp)
    - OPR: highest (scan future references every fault)
3. **Belady's Anomaly (FIFO issue)**
    - FIFO can sometimes produce **more faults when frames increase.**
    - LRU and OPR are **stack algorithms, so faults** do not increase with more frames for the same reference string.
4. **Real OS practicality**
    - OPR cannot be used in real OS (future unknown).
    - LRU is expensive if implemented perfectly; OS often uses approximations (Clock/Second Chance, aging, etc.)
    - FIFO is cheap but often poor.

7.SUMMARY

- FIFO is easy but can be inefficient and can show Belady’s anomaly.
- LRU improves performance by using recency; often close to optimal.
- OPR is best possible theoretically and used as benchmark, not in real systems.
- Practical OS designs prefer LRU-like approximations to balance performance and overhead.